Tema 8. Gestión del estado en frameworks JS

o cómo dejar de arreglar las cosas reiniciando

Hoy

1. Qué es el estado
2. Estado en el árbol
3. Estado global de cliente
4. Estado del servidor
5. En resumen

Bugs que seguro habéis visto/tenido

- La cabecera dice “**3 pendientes**” y la lista muestra 2
- Marcas un item como comprado, recargas… y vuelve a estar sin comprar
- Dos pestañas abiertas, dos versiones distintas del mismo listado

Siempre es el mismo problema

Dos copias del mismo dato que se han desincronizado

De hecho, un *bug* de este tipo en Facebook dio origen a la “arquitectura Flux”, una idea que está en el origen de la “gestión centralizada del estado” (apartado 3)

“80% of all problems in all complex systems are fixed by rebooting.”

Stuart Halloway, junio 2019

Reiniciar = volver a un **estado** conocido

8.1 ¿Qué es el estado?

El estado de una app

> *El conjunto de datos **no derivables** que gobiernan lo que muestra la interfaz*

- Datos que vienen del servidor
- Datos introducidos por el usuario pendientes de sincronizar
- Filtros locales (ver solo los items pendientes)
- Información global: usuario autenticado, preferencias de UI, ruta activa…

UI = f(estado)

La vista es una función del estado. El framework (reactividad) se encarga de recalcularla cuando el estado cambia. **Nosotros solo decidimos qué es estado y dónde vive.**

Estado vs. estado derivado

❌ Guardar algo que se puede calcular

```
const items = ref([...])
const pendientes = ref(0)

function toggle(item) {
  item.comprado = !item.comprado
  // ¿y si nos olvidamos de hacer
esto?
  pendientes.value += item.comprado
? -1 : 1
}
```

✅ Calcularlo a partir del estado

```
const items = ref([...])

//no podemos olvidarnos, se hace
automáticamente
const pendientes = computed(() =>
  items.value.filter(i =>
!i.comprado).length
)
```

Regla: si se puede calcular, no es estado → `computed` (tema de reactividad)

En la lista de la compra, ¿es estado?

1. La lista de productos
2. El número de productos pendientes
3. El texto del input “añadir producto” mientras se escribe
4. El filtro “solo mostrar pendientes” activado/desactivado
5. La lista de productos filtrada
6. El usuario autenticado
7. Qué item se está editando ahora mismo
8. Tema claro/oscuro

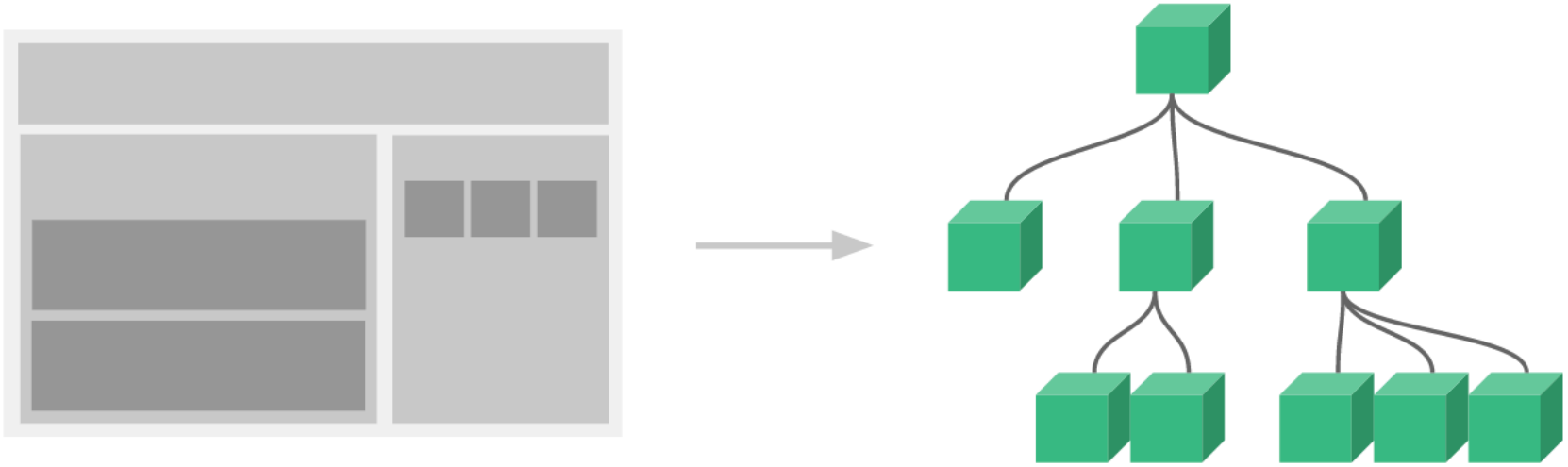
No todo el estado es igual

Tipo	Ejemplo	Dónde vive
Local de UI	texto de un input, modal abierto	el propio componente
Compartido de cliente	filtro, tema oscuro	ancestro común · *provide/inject* · *store*
De servidor	lista de productos	caché de servidor (TanStack Query…)
Persistente local	preferencias	*store* con `localStorage`

Vamos a ir viendo las distintas opciones

8.2 El estado en el árbol de componentes

Recordad que las apps son árboles de componentes



¿Qué componentes guardan estado? ¿Quién guarda qué?

Primera posibilidad: cada componente guarda su estado

```
<script setup>
import { ref } from 'vue'
const props = defineProps(['id', 'nombre', 'comprado'])
const comprado = ref(props.comprado)
const toggle = () => { comprado.value = !comprado.value }
</script>

<template>
  <li :class="{ tachado: comprado }" @click="toggle">{{ nombre }}</li>
</template>
```

Problema: ¿cómo contaría un componente `Resumen` los items pendientes? ¿Cómo filtramos u ordenamos?

El estado introduce complejidad

Un componente **sin estado** o *presentacional* es una función pura: props → HTML

```
<script setup>
defineProps(['id', 'nombre', 'comprado'])
defineEmits(['toggle', 'borrar'])
</script>

<template>
  <span :class="{ tachado: comprado }"
        @click="$emit('toggle', id)">
    {{ nombre }}
  </span>
  <button @click="$emit('borrar',
id)">Eliminar</button>
</template>
```

Componentes **presentacionales**:

- Más reutilizables
- Más fáciles de testear
- Sin efectos laterales: menos *debugging*

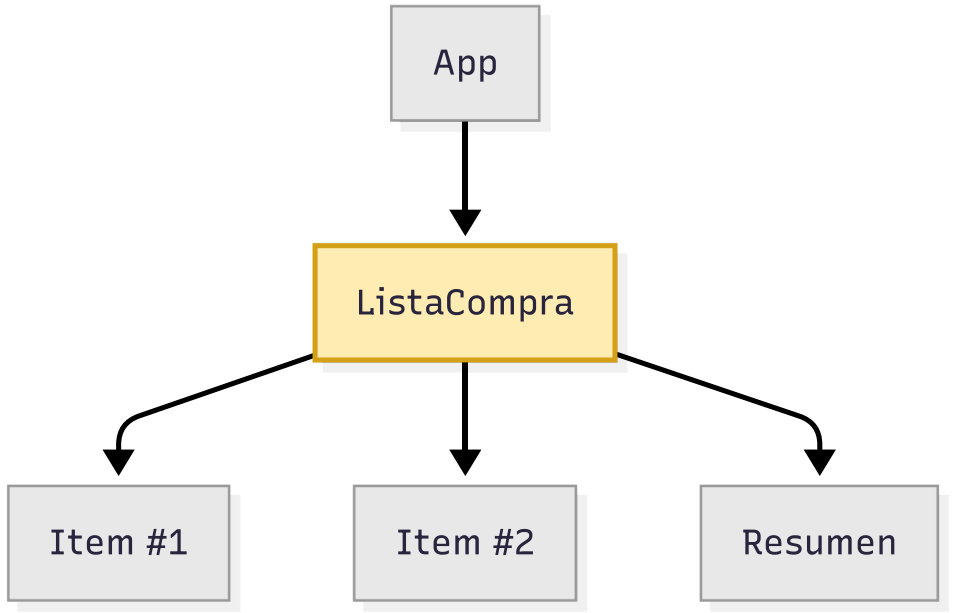
Objetivo

Muchos componentes presentacionales y pocos con estado. Pero ¿cuáles?

Levantar el estado (*lifting state up*)

> *El estado debe subir al **ancestro común más cercano** de los componentes que lo necesiten*

Se formuló en React, pero hoy lo asumen todos los *frameworks*

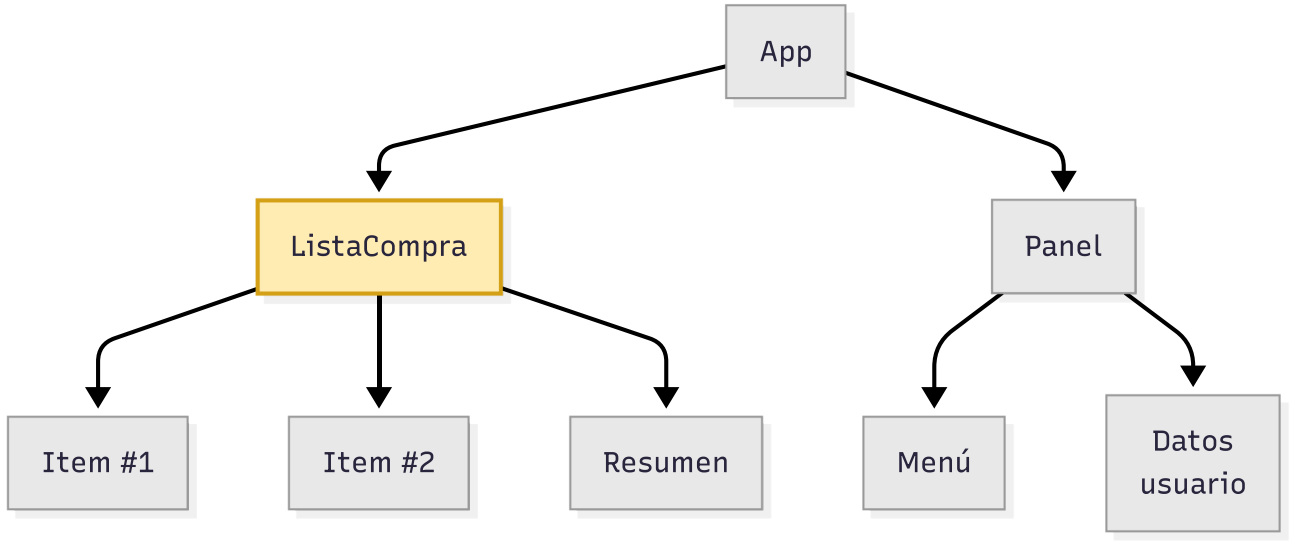


Al ancestro común más cercano

No es “cuanto más alto, mejor”:

- Más arriba = más componentes se re-renderizan
- Más arriba = más niveles por los que pasar *props*

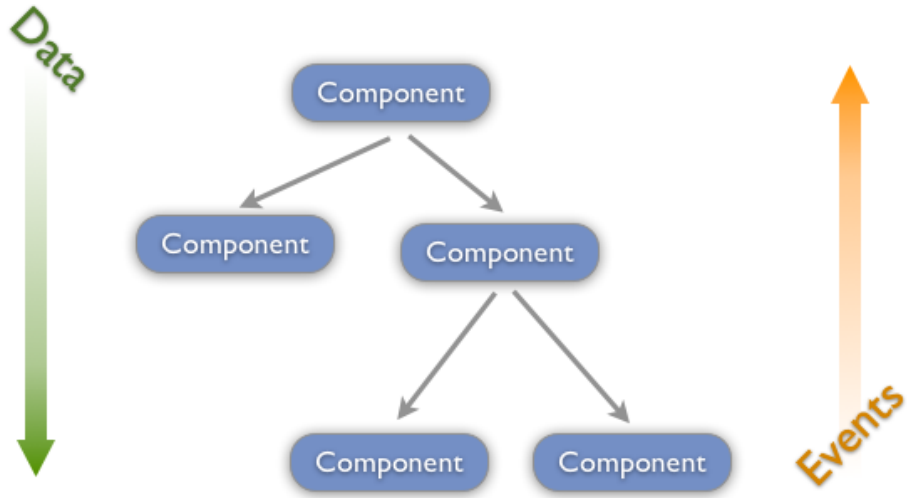
`Panel` no necesita la lista: no hay razón para subirla a `App`



Flujo de datos unidireccional

El ancestro es la **fuente de verdad**:

1. El ancestro tiene el estado
2. Los datos **bajan** como *props*
3. Los hijos no cambian el estado: **suben** eventos



El ancestro con estado, en código

```
<!-- ListaCompra.vue -->
<script setup>
const items = ref([])
const pendientes = computed(() => items.value.filter(i => !i.comprado).length)

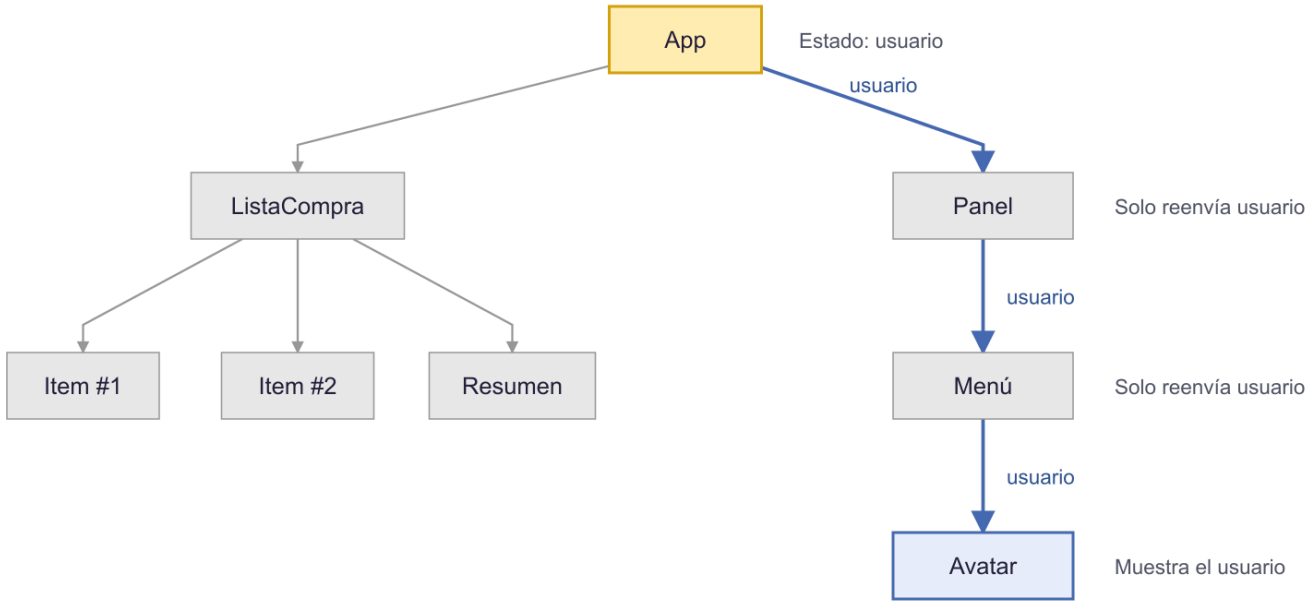
function toggle(id) {
  const item = items.value.find(i => i.id === id)
  item.comprado = !item.comprado
}
</script>

<template>
  <Item v-for="it in items" :key="it.id" v-bind="it"
        @toggle="toggle" @borrar="borrar" />
  <Resumen :pendientes="pendientes" />
</template>
```

Cuando el árbol crece: *prop drilling*

Queremos mostrar el usuario en un `Avatar` dentro del `Menú`

- `App` tiene el usuario
- `Panel` y `Menú` tienen que recibir la *prop* solo para poder pasarla hacia abajo
- Los componentes intermedios quedan acoplados a un dato que no usan



En Vue: `provide` / `inject`

```
// App.vue: el dueño sigue siendo App
const usuario = ref(null)
//readonly devuelve un proxy de solo lectura, si modificamos algo dará error/no
funcionará
provide('usuario', readonly(usuario))
provide('logout', () => { usuario.value = null })
```

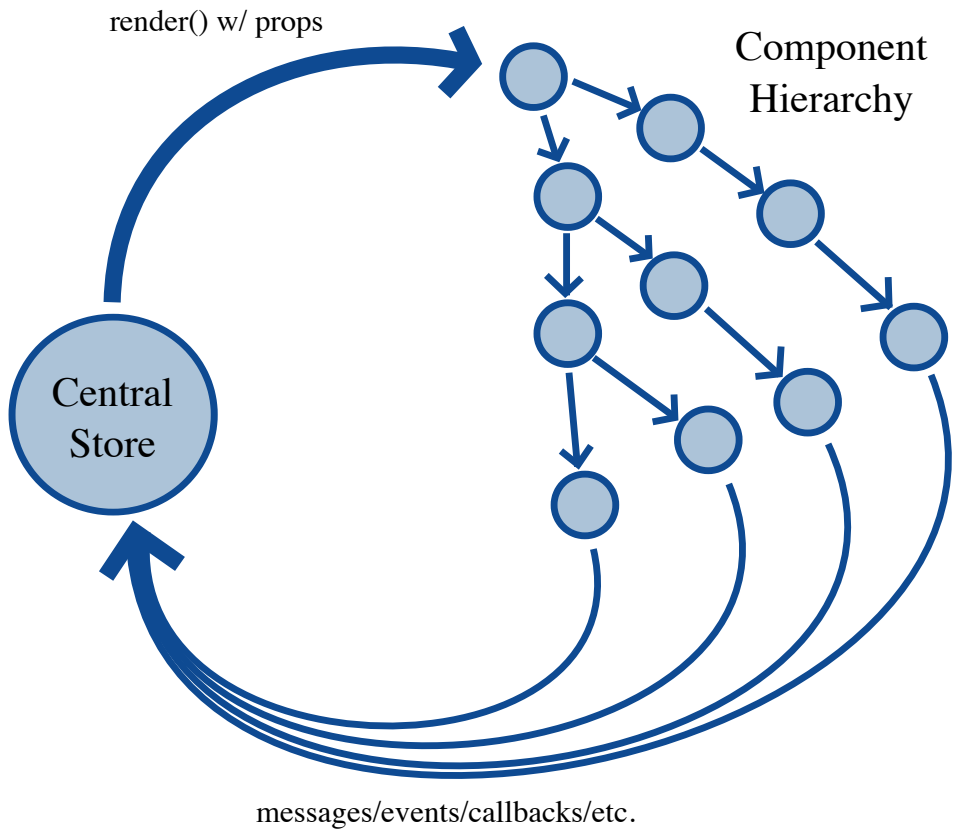
```
// Avatar.vue: a cualquier profundidad
const usuario = inject('usuario')
const logout = inject('logout')
```

- Se salta los niveles intermedios pero solo lo pueden recibir los descendientes
- Mantiene el flujo unidireccional
- En React: `createContext` / `useContext`

8.3 Estado global de cliente: el patrón *store*

Levantar el estado “versión extrema”

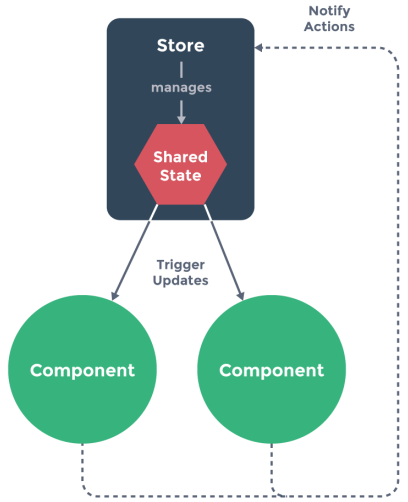
- Sacar el estado fuera del árbol: un **almacén centralizado** (**store**) con el estado compartido
- Los componentes lo leen y le piden cambios
- Casi todos los componentes pasan a ser presentacionales



Un *store* "casero"

```
// store.js: módulo = al importar, una única instancia
import { reactive } from 'vue'

export const store = {
  state: reactive({ message: 'Hello!' }),
  setMessage(nuevo) { this.state.message = nuevo },
  clearMessage() { this.state.message = '' }
}
```



- Los componentes **no modifican directamente** el estado
- Los cambios pasan por **métodos del *store***

Ejemplo online

Implementación propia vs librería estándar

Aunque ya hemos visto que es muy sencillo implementar de cero un *store*, en Vue se usa habitualmente una librería “estándar”: **Pinia**

Ventajas:

- Integración con las *devtools*
- Integración con SSR (Server Side Rendering, lo veremos en el próximo tema)
- *Plugins*
- Utilidades de *testing*

De dónde viene Pinia

Año		
2012	Arquitectura Elm	estado + mensajes + función de actualización
2014	Flux (Facebook)	patrón de flujo unidireccional para React
2015	Redux	la implementación de Flux de más éxito
2016	Vuex	Redux para Vue, menos funcional (hoy en mantenimiento)
2021	Pinia	Vuex simplificado; el *store* oficial de Vue

Mismas ideas en otros *frameworks*: Redux/Zustand (React), NgRx (Angular)…

Un *store* de Pinia

```
export const useContadorStore =
 defineStore('contador', () => {
    const valor = ref(0)
    const valorDoble = computed(
      () => valor.value * 2
    )
    function incrementar() { valor.value++ }

    return { valor, valorDoble, incrementar }
})
```

Primitiva Vue/JS	En Pinia	Significado
`ref`	state	El estado
`computed`	getter	Valores derivados
`function`	action	Lógica de negocio

Usar el *store* en un componente

```
<script setup>
import { useContadorStore } from './store.js'

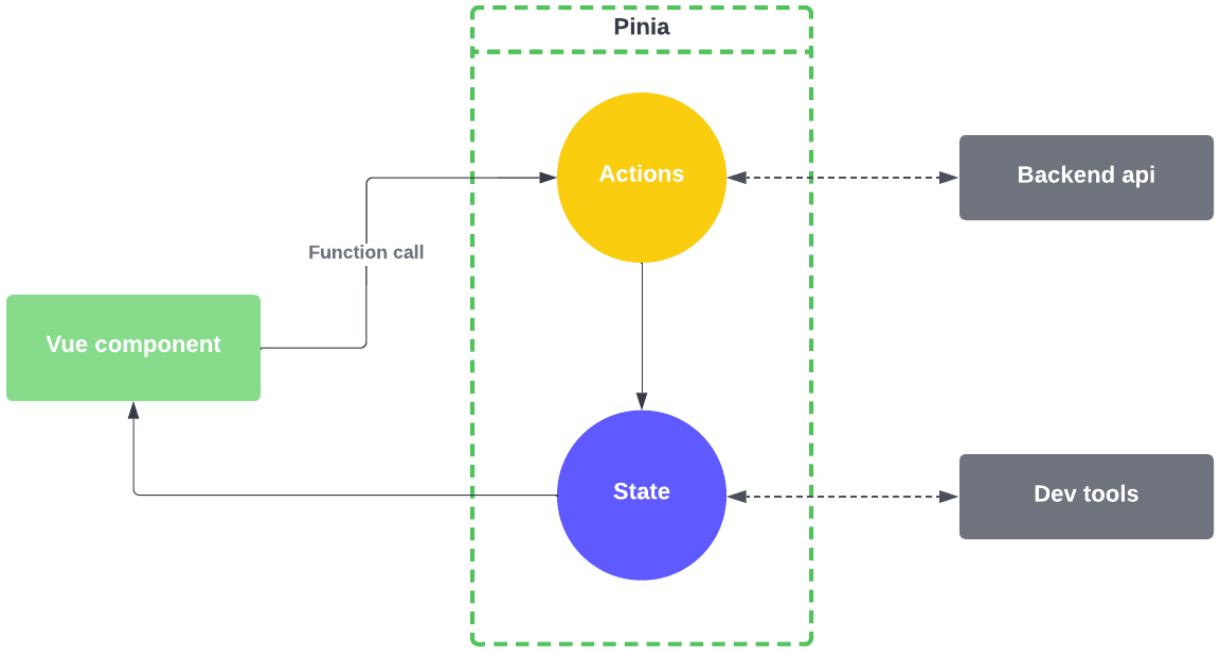
const store = useContadorStore()
</script>

<template>
  <h1>{{ store.valor }}</h1>
  <button @click="store.incrementar()">Incrementar</button>
</template>
```

Aclaración: el componente puede mutar el estado: `store.valor++` pero es buena práctica hacerlo a través de una acción

El ejemplo *online*

Flujo unidireccional en Pinia

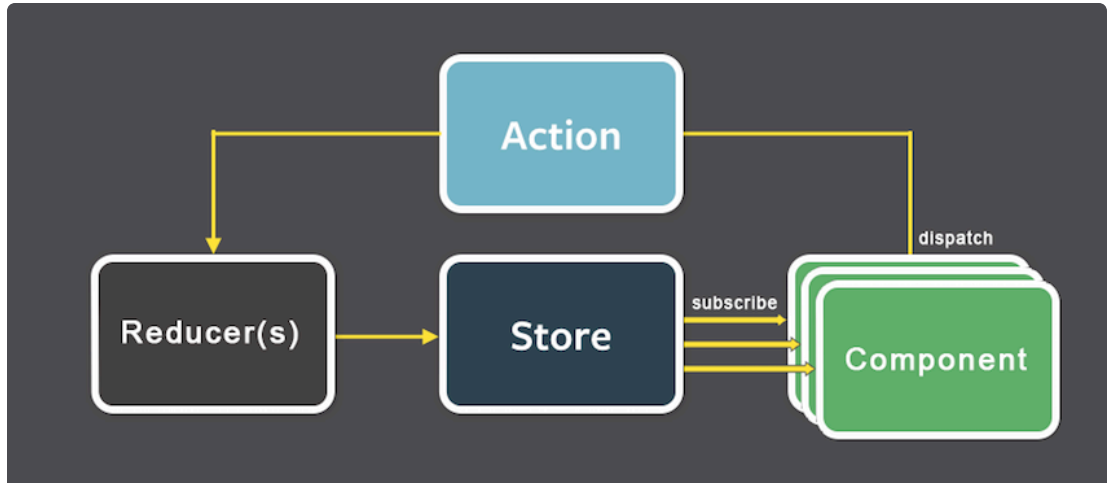


Lista de la compra con Pinia y sin *backend* (con un *plugin* que persiste el estado en localStorage)

Mutable vs. inmutable: Pinia vs. Redux

Pinia: la *action* **muta** el estado; los *proxies* detectan el cambio

Redux: un *reducer* puro **devuelve un estado nuevo**; se detecta el cambio comparando referencias



La diferencia viene del **modelo de reactividad** de cada *framework*

Hoy se usa Redux Toolkit, que con Immer permite escribir “como si” mutaras

Zustand (React)

```
// store.js
import { create } from 'zustand'

export const useCounterStore = create((set) => ({
  count: 0,
  increment: () =>
    set((state) => ({ count: state.count + 1 })),
}))
```

```
// App.jsx
import { useCounterStore } from './store.js'

export default function App() {
  const count = useCounterStore((s) => s.count)
  const increment = useCounterStore((s) => s.increment)
  return <button onClick={increment}>{count}</button>
}
```

- Simplificación de Redux, como Vuex → Pinia
- `set` devuelve un objeto nuevo: inmutabilidad, como en Redux

Ejemplo *online*

Svelte y SolidJS

- **Svelte 5**: los stores siguen existiendo, pero lo recomendado es un `$state()` exportado desde un módulo `.svelte.js`: un *store* "casero" con *runes*
- **SolidJS**: *signals* para lo simple y stores para estado anidado; algo intermedio entre Svelte y Pinia

Con reactividad de grano fino, un *store* puede ser simplemente **estado reactivo fuera de los componentes**

¿Necesito un *store* global?

> *“People often choose Redux before they need it.”*

Dan Abramov, You Might Not Need Redux

Sí, si:

- Muchos componentes alejados leen y escriben lo mismo
- Necesitas devtools, persistencia o SSR

Si no:

- Estado local + *lifting* + provide/inject
- Y para datos del servidor… **siguiente parte**

8.4 Estado del servidor

Client state vs. *server state*

Client state

- Lo crea el usuario en local
- Es **tuyo**: nadie más lo cambia
- Disponible al instante

Ej: lista de la compra local, filtros, tema

Server state

- Vive en el servidor (p.ej. Supabase)
- **No es tuyo**: tienes una copia en caché. Otros lo pueden cambiar en segundo plano
- Tarda en llegar, puede fallar

Ej: lista de la compra colaborativa

En el caso de la lista de la compra “monousuario”, ¿la propia lista es *client* o *server* state?

El criterio decisivo es **quién tiene la “fuente de verdad”**

Lo que hay que gestionar con el *server state*

- Estado de **carga** (*loading*)
- **Errores** y reintentos
- **Caché**: no pedir lo mismo dos veces
- **Datos obsoletos**: cuándo volver a pedir
- **Duplicación**: dos componentes piden lo mismo a la vez
- **Sincronizar** tras modificar: ¿qué datos quedan desfasados?

Pinia no está pensada para esto

A mano

```
const posts = ref([])
const loading = ref(false)
const error = ref(null)

onMounted(async () => {
  loading.value = true
  try {
    posts.value = await getPosts()
  } catch (e) {
    error.value = e
  } finally {
    loading.value = false
  }
})
```

- Cada componente hace su `fetch`
- *loading*/*error* duplicados en todos
- Sin caché ni revalidación
- Peticiones repetidas si varios componentes muestran lo mismo
- Y pueden quedar **desincronizados**: cada uno tiene su copia de los datos

TanStack Query

```
import { useQuery } from '@tanstack/vue-query'

const { data, isPending, isFetching, isError, error
} = useQuery({
  queryKey: ['posts'],
  queryFn: getPosts,
  staleTime: 10000,   //ms
})
```

- `isPending` : todavía no hay datos
- `isFetching` : hay una petición en curso (aunque ya haya datos)

✅ *loading*/*error* reactivos

✅ Caché compartida por `queryKey`

✅ No duplica peticiones

✅ *Refetch* automático (al volver a la pestaña, al reconectar…)

tanstack.com/query · Para Vue, React, Svelte, Solid, Angular

Demo *online*

La caché por `queryKey`

- La `queryKey` es el **nombre** del dato en la caché
- Dos componentes con la misma clave → **una sola petición** y los mismos datos, sin props ni *store*
- Si el dato depende de un parámetro, el parámetro **va en la clave**: cada valor es una entrada distinta de la caché

```
const page = ref(1)

const { data } = useQuery({
  // si cambia page, cambia la clave
  queryKey: ['posts', 'page', page],
  queryFn: () =>
getPostsPage(page.value),
  // mientras llega la página nueva,
mostrar la anterior
  placeholderData: keepPreviousData,
})
```

Mutaciones

- Para **modificar** datos: `useMutation`
- Después hay que avisar a la caché de que hay datos desfasados: se **invalida** la clave
- Las queries con esa clave (o que **empiezan** por ella) pasan a *stale* y las que están en pantalla se vuelven a pedir

```
const qc = useQueryClient()

const { mutate } = useMutation({
  mutationFn: likePost,
  onSuccess: () =>
qc.invalidateQueries({ queryKey:
['posts'] }),
})

mutate(post.id)
```

Actualizaciones optimistas

1. Antes de la respuesta, se **cancelan** los *refetch* en curso y se actualiza la **caché local**: la interfaz parece instantánea
2. Si el servidor da error (`onError`), se **deshace** el cambio
3. Al terminar (`onSettled`), se **invalida** para quedarse con lo que diga el servidor (por si otros han cambiado algo)

```
useMutation({
  mutationFn: likePost,
  onMutate: async (id) => {
    await qc.cancelQueries({ queryKey: ['posts'] })
    const anterior = qc.getQueryData(['posts'])
    qc.setQueryData(['posts'], (posts) =>
      posts.map(p => p.id === id ? { ...p, likes: p.likes + 1 }
: p))
    return { anterior }
  },
  onError: (_error, _id, contexto) =>
    qc.setQueryData(['posts'], contexto.anterior),
  onSettled: () =>
    qc.invalidateQueries({ queryKey: ['posts'] }),
})
```

Solo compensa si la espera se nota mucho: obliga a repetir en el cliente la lógica del servidor. `useLikeOptimista` en el ejemplo

Alternativas y extras

- Pinia Colada: lo mismo que TanStack Query, pensado para Vue
- **SWR** (React), **RTK Query** (Redux), **Apollo** (GraphQL)
- **Tiempo real** (Supabase Realtime): cuando llega un cambio, actualizar o invalidar la caché (`setQueryData` / `invalidateQueries`)

8.5 En resumen

Árbol de decisión

1. ¿Se puede **calcular**? → `computed`: no es estado
2. ¿Viene del **servidor**? → caché de servidor (TanStack Query, Pinia Colada)
3. ¿Debe sobrevivir a una recarga o **compartirse por enlace**? → URL / localStorage
4. ¿Lo usa **un solo** componente? → estado local
5. ¿Lo usan **unos pocos cercanos**? → ancestro común más cercano
6. ¿Lo usan **descendientes lejanos** de un mismo componente? → provide/inject
7. ¿Lo usa **toda la app**? → *store* (Pinia)

Actividad: diseñad el estado

Lista de la compra **colaborativa** con:

- Panel con el usuario y su avatar
- Lista de items compartida por la familia
- Filtro “solo pendientes”
- Contador de pendientes
- Modal para editar un item
- Tema claro/oscuro

Por parejas: **¿dónde vive cada dato?**

Resumen

- El estado son los datos **no derivables**; lo derivado se calcula
- Hay **tipos** de estado y cada uno tiene su sitio
- En el árbol: **ancestro común más cercano**, flujo unidireccional, provide/inject para saltar niveles
- *Store* global solo cuando de verdad hace falta; Pinia = reactividad + convención
- Los datos del servidor son una **caché**: mejor una librería que hacerlo a mano